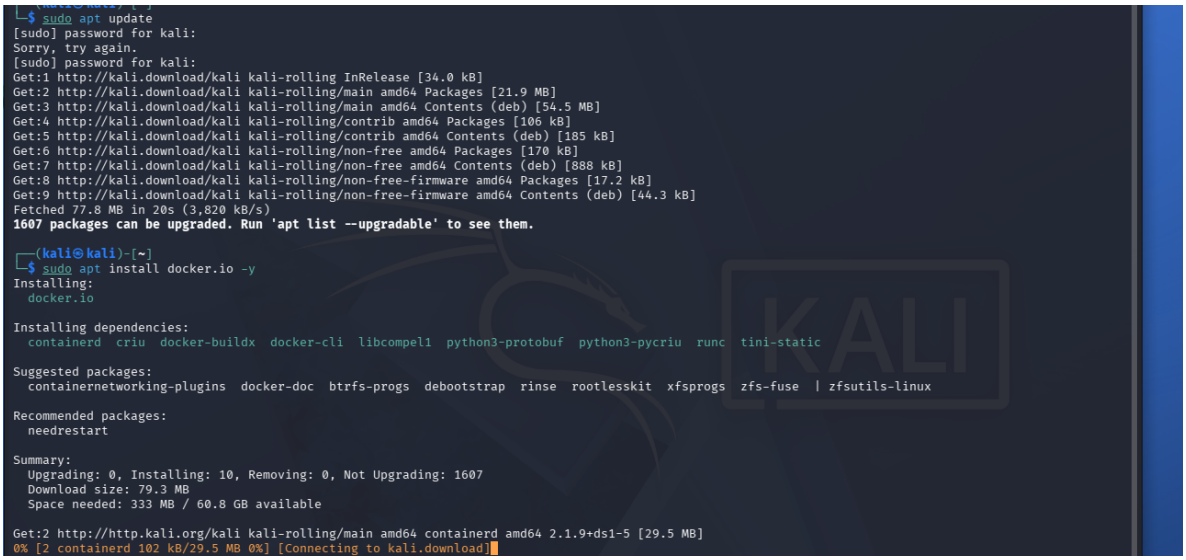
LABORATORIO DVWA – Parte 1 Damn Vulnerable Web Application

PARTE 1: Preparar el entorno en Kali Linux

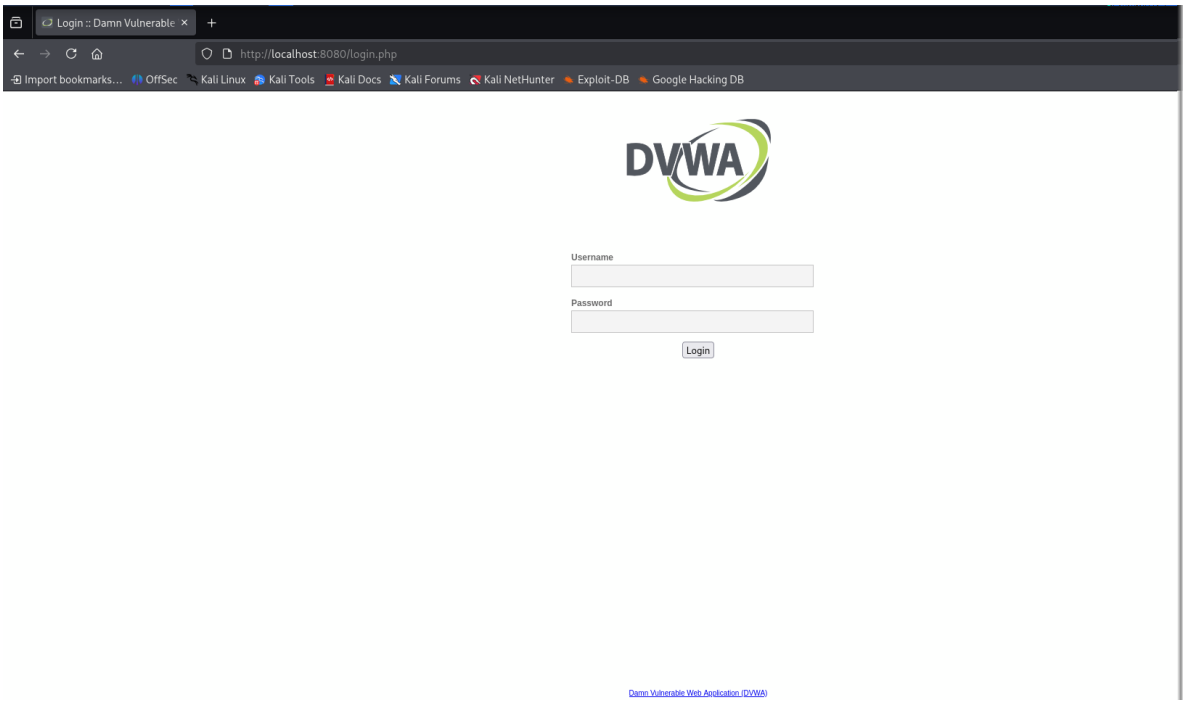




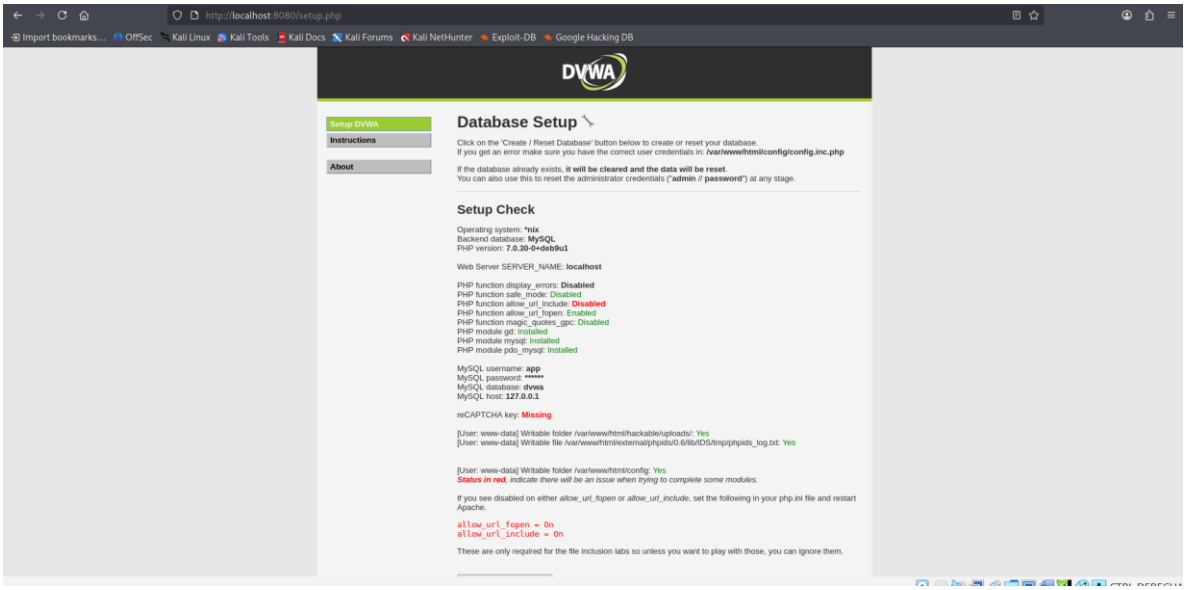
PARTE 2: Ejecutar DVWA



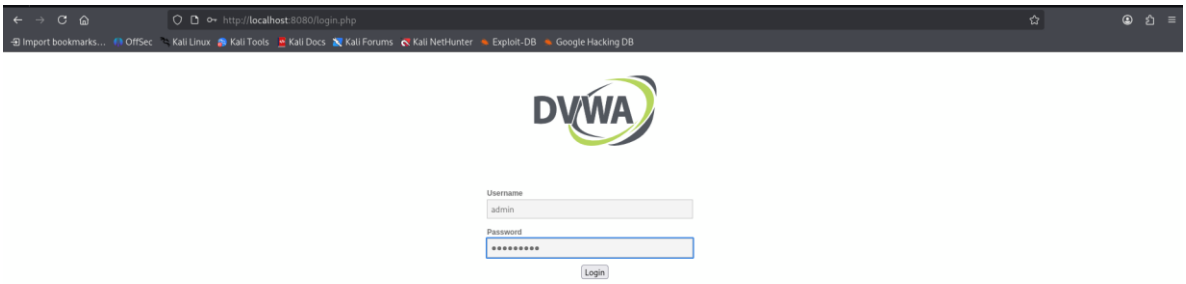
PARTE 3: Abrir DVWA en el navegador



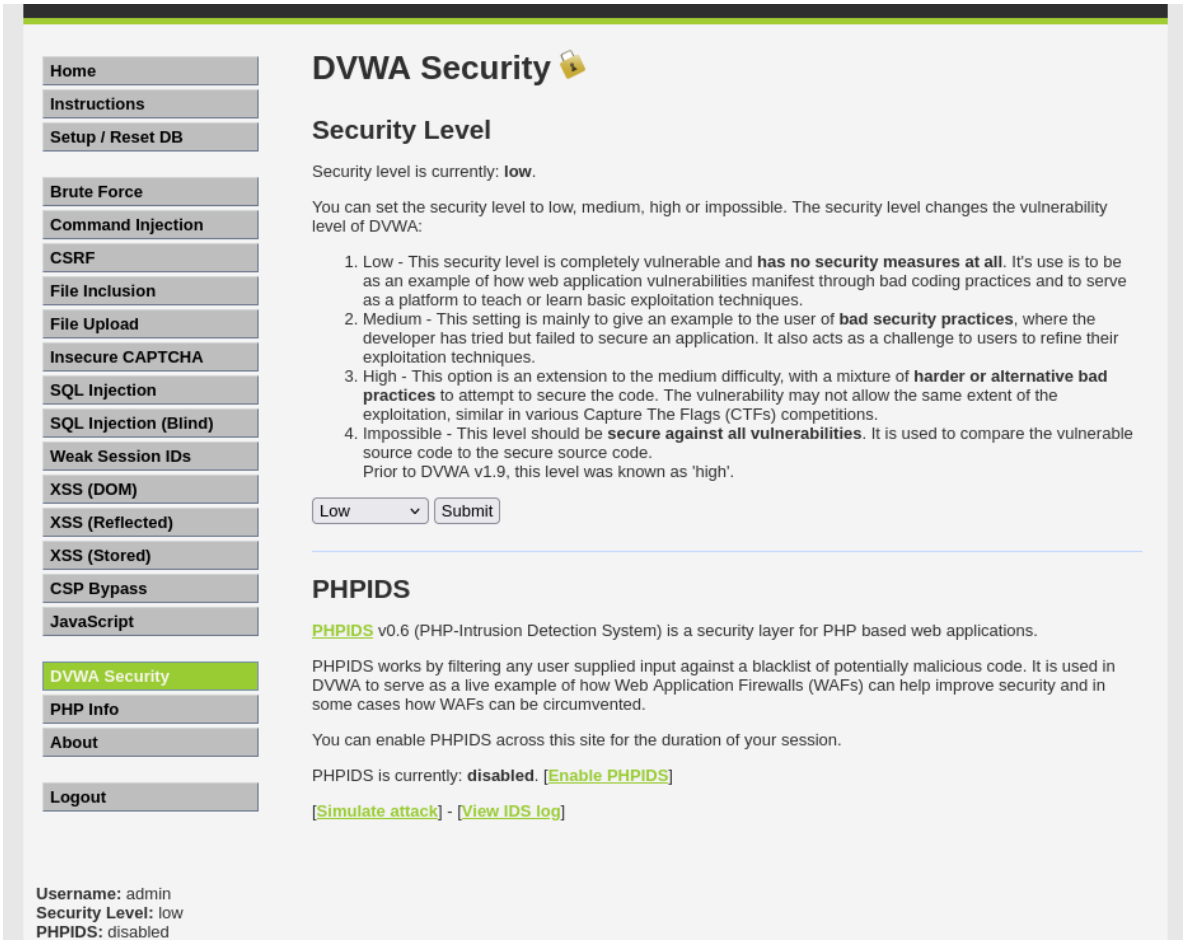
PARTE 4: Iniciar sesión

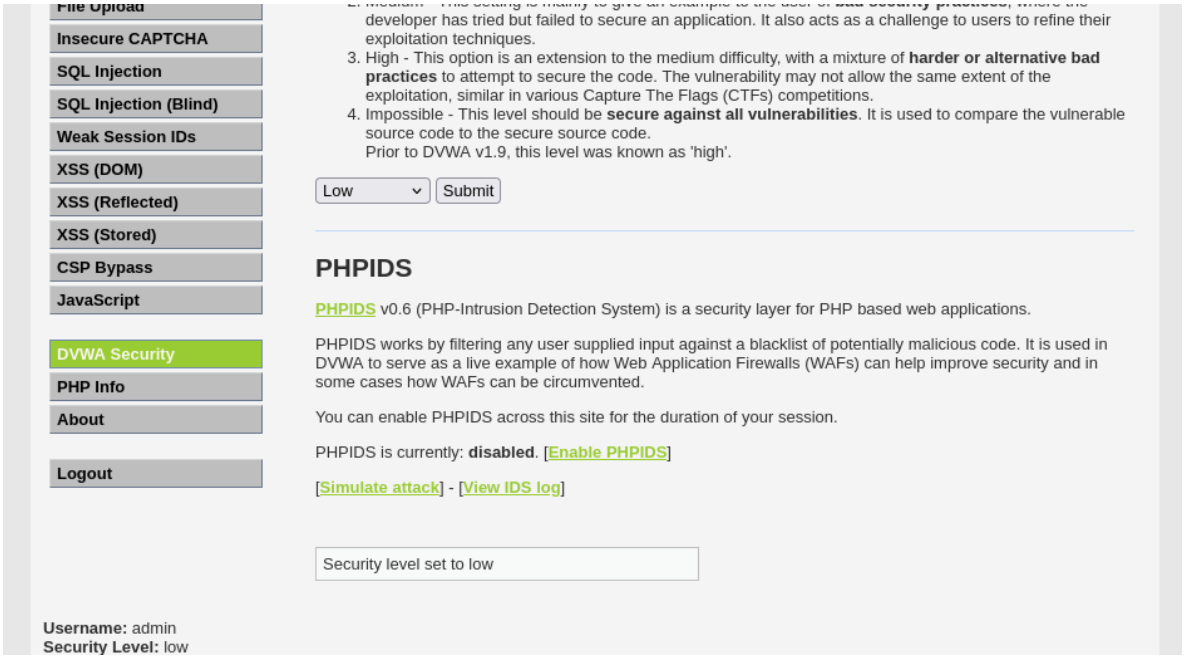


PARTE 5: Crear o restaurar la base de datos



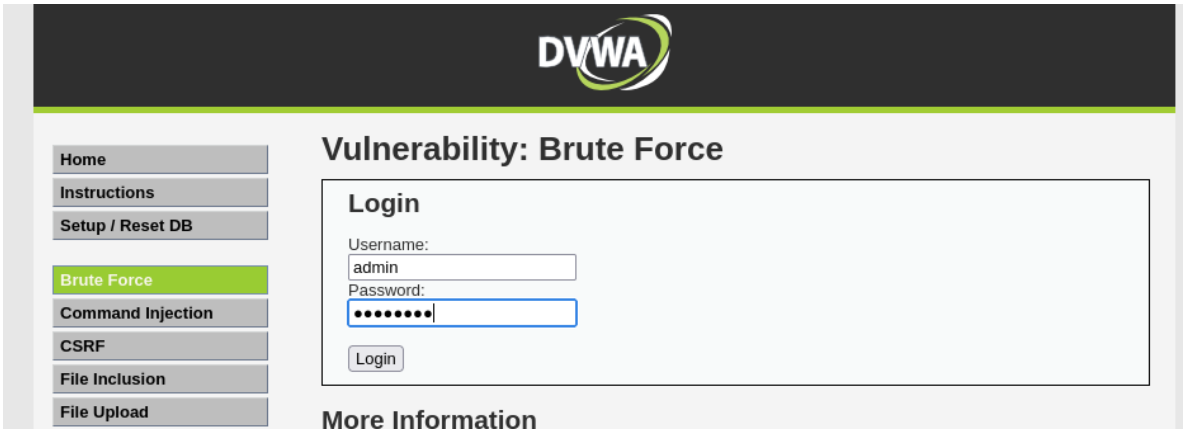
PARTE 6: Configurar el nivel de seguridad

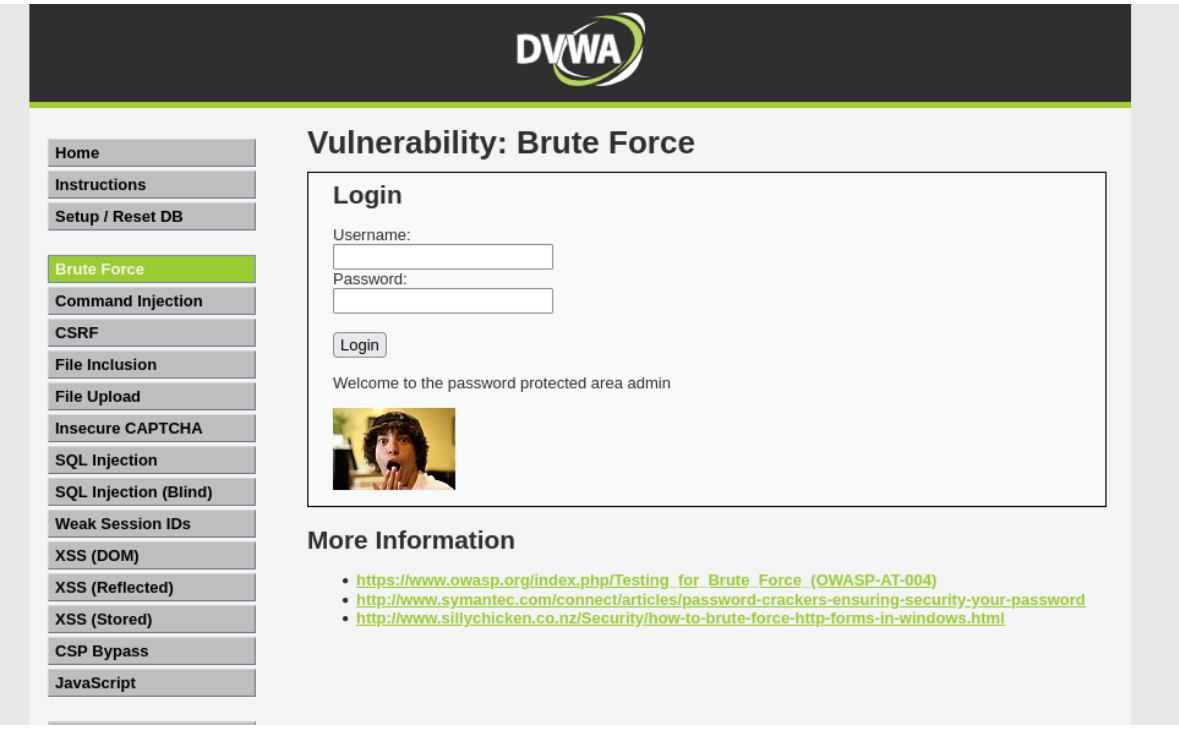




MÓDULOS DE TRABAJO

PARTE 7: Brute Force



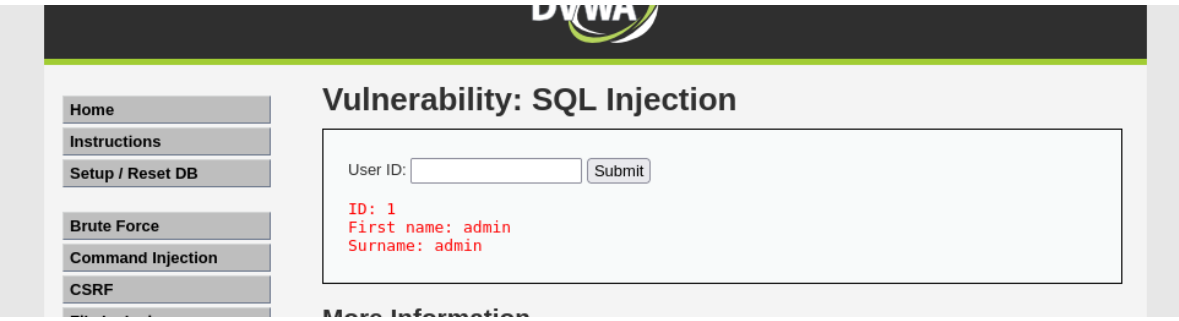


Conclusión técnica: Al analizar el comportamiento del sistema tras el ingreso exitoso, se evidencia que el formulario de autenticación procesa las peticiones a través del método GET (haciendo que las credenciales viajen expuestas en la URL) y carece de mecanismos de control de flujo. Al no existir un límite de intentos fallidos ni bloqueos de sesión, el servidor procesa peticiones continuas sin restricciones, dejando la puerta abierta para que herramientas automatizadas adivinen contraseñas mediante fuerza bruta o ataques de diccionario.

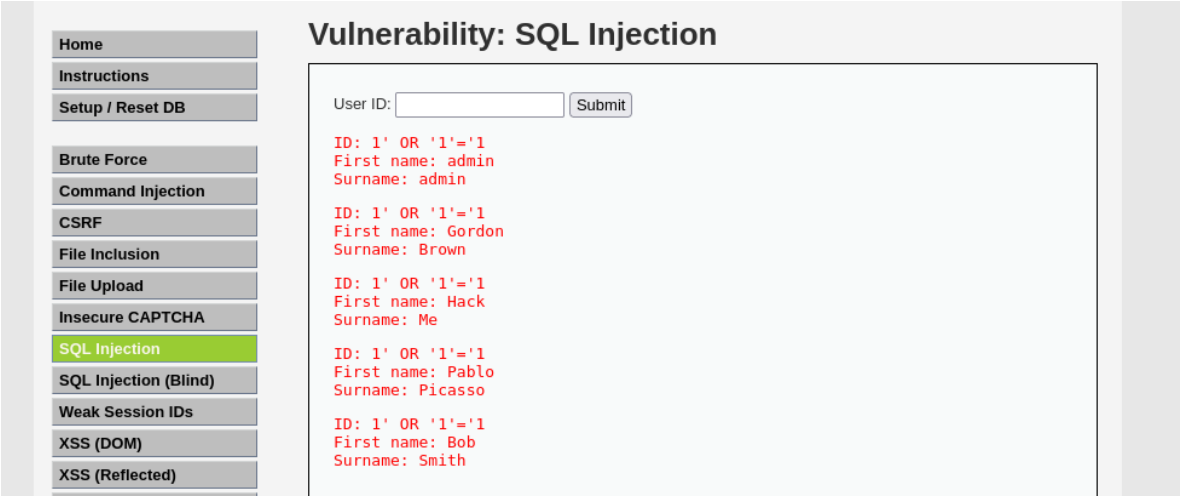
Mitigación: Para solucionar esta vulnerabilidad, primero es necesario cambiar la transmisión de datos mediante POST para ocultar las credenciales de la URL. Además, se debe implementar un sistema de bloqueo temporal de cuenta, es decir, 10 minutos tras x intentos fallidos, lo que haría inviable y tedioso el uso de scrpits automatizados

PARTE 8: SQL Injection

Prueba normal (comportamiento esperado)



Prueba de inyección básica



Forzar error (prueba de validación)

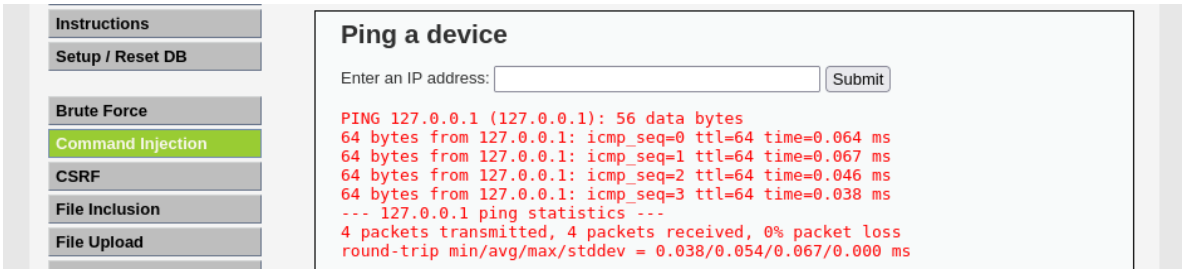


Conclusión técnica: Las evidencias visuales demuestran que el backend concatena el valor ingresado en el campo "User ID" directamente en la cadena de la consulta SQL original, sin realizar ninguna limpieza o validación estructural. Al ingresar secuencias lógicas como 1' OR '1'='1, el motor de base de datos evaluó la condición como verdadera para todos los registros de la tabla, confirmando que la entrada del usuario está logrando manipular la lógica interna de la consulta.

Mitigación: Se debería evitar el uso de listas negras con caracteres prohibidos, ya que son fáciles de evitar. Lo mejor que se puede hacer es utilizar sentencias preparadas o consultas parametrizadas, así, el motor de la base de datos recibe la entrada del usuario como un valor simple, no como un comando SQL.

PARTE 9: Command Injection

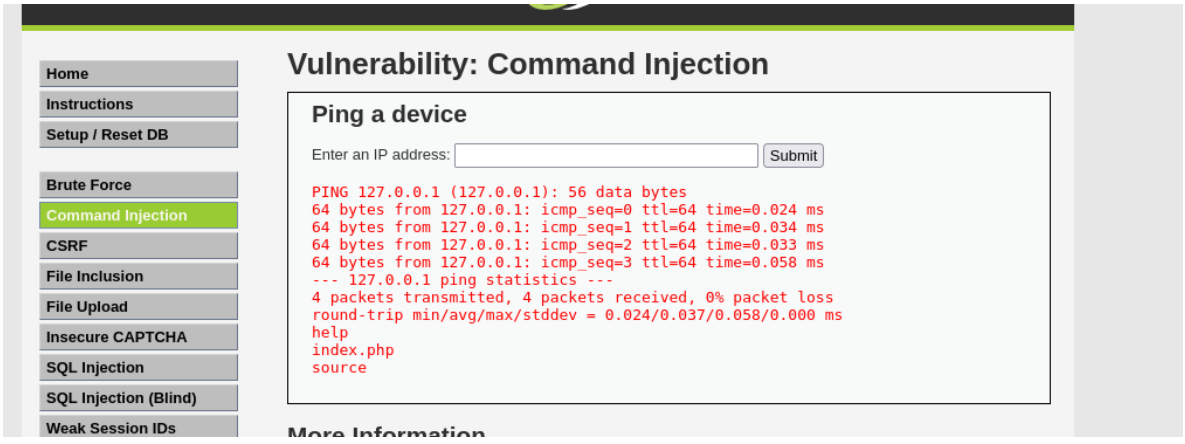
Prueba normal (comportamiento esperado)



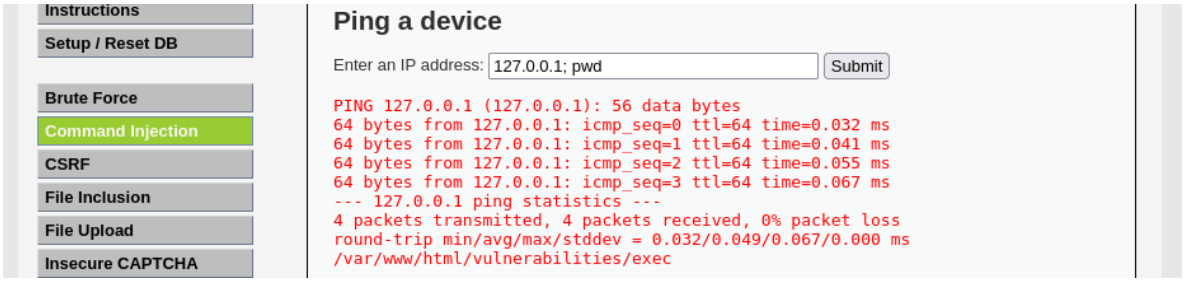
Primer intento de inyección



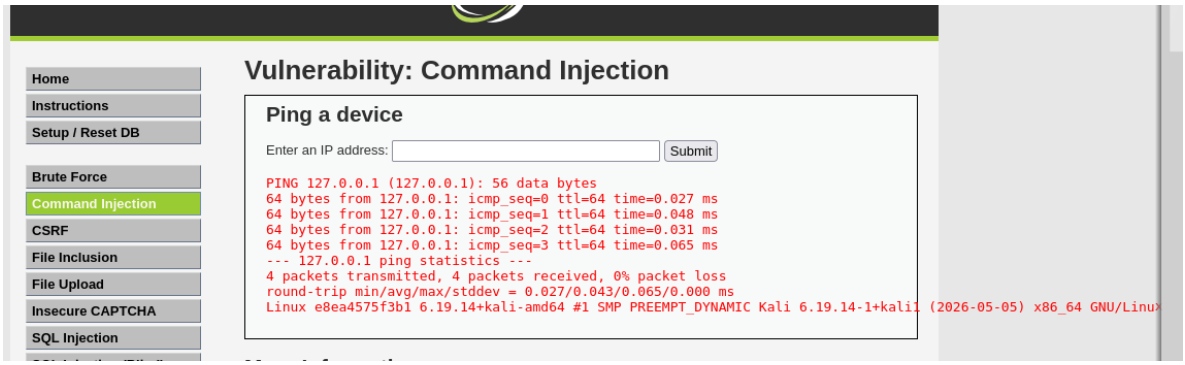
Listar archivos (LS)



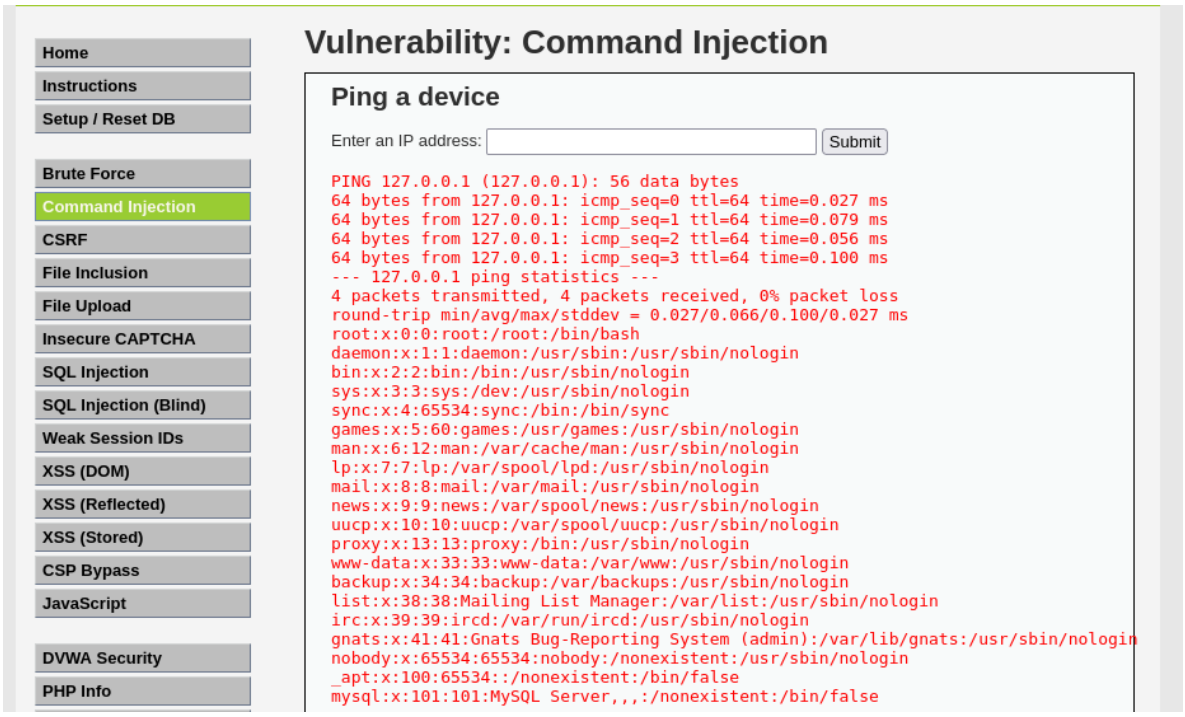
Ver ruta actual (PWD)



Información del sistema



Leer archivo del sistema

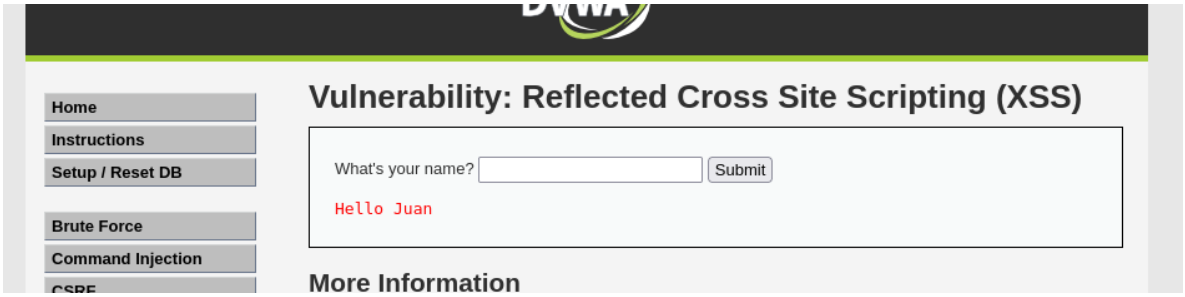


Conclusión técnica: Las capturas donde se muestran los resultados de los comandos uname -a y cat /etc/passwd revelan un fallo crítico: la aplicación pasa el input del usuario directamente a una función que ejecuta comandos del sistema operativo (como exec o system). El hecho de que el carácter punto y coma (;) haya permitido encadenar instrucciones adicionales al comando "ping" original, demuestra la total ausencia de un filtro que valide el formato del dato recibido.

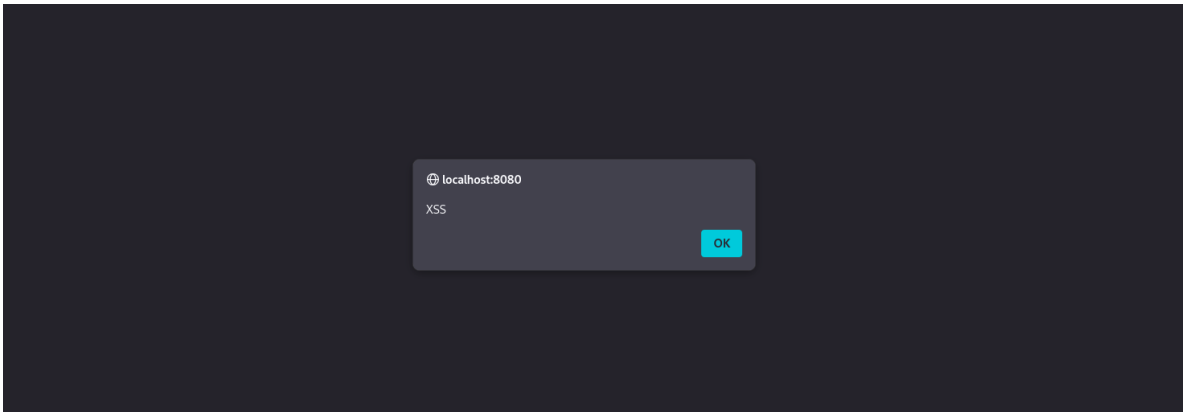
Mitigación: Considero que el enfoque más seguro es aplicar el principio de lista blanca (allowlist) mediante una validación estricta de formato. Antes de que el dato toque alguna función del sistema, el código backend debe verificar mediante una expresión regular (Regex) o una función nativa de validación que el input corresponda exactamente al patrón de una dirección IPv4. Si la cadena contiene cualquier carácter ajeno a números y puntos, la petición debe descartarse inmediatamente.

PARTE 10: XSS Reflected

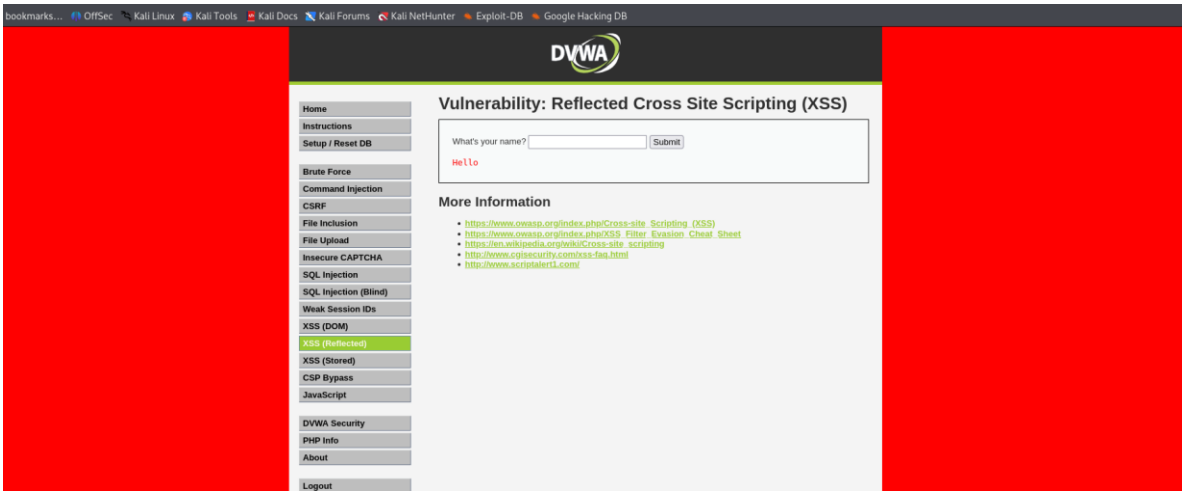
Prueba normal (comportamiento esperado)



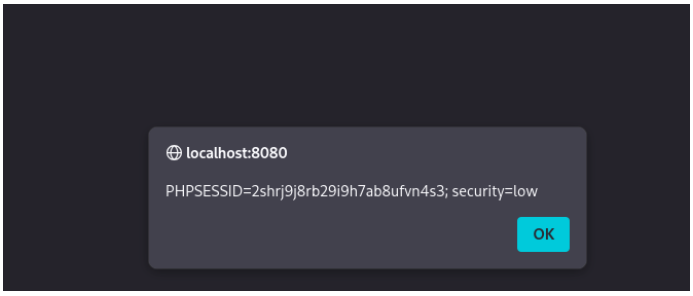
Inyección básica (primer XSS)



Modificar la página (prueba visual)



Acceso a cookies (nivel real)



Conclusión técnica: Tal como se observa en la captura donde se modifica el estilo visual de la página y salta la ventana emergente, la aplicación toma el nombre enviado por el usuario y lo incrusta de inmediato en el código HTML de respuesta. Al no existir un proceso de sanitización, el navegador del cliente no puede distinguir entre la estructura legítima de la página y el script inyectado, procediendo a ejecutar las etiquetas <script> de manera automática en el contexto del documento actual.

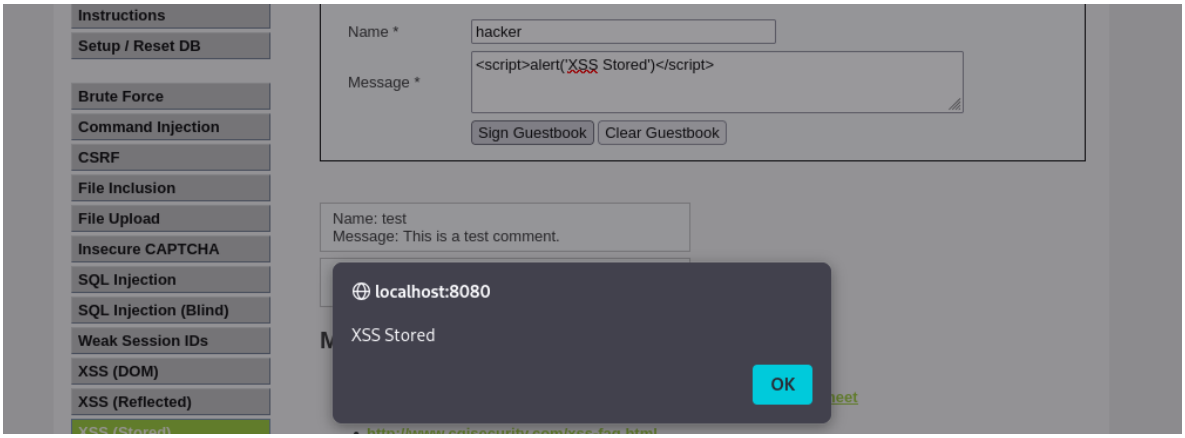
Mitigación: Para evitar que el navegador interprete estas entradas como código, aplicaría un proceso de codificación de salida (Output Encoding) justo antes de imprimir la variable en el frontend. Usando funciones nativas (como htmlspecialchars), transformaría los caracteres especiales como < y > en sus entidades HTML correspondientes (<, >). Así, el navegador solo renderiza visualmente el código malicioso como simple texto inofensivo, neutralizando su ejecución.

PARTE 11. XSS (Stored) – Paso a paso (DVWA)

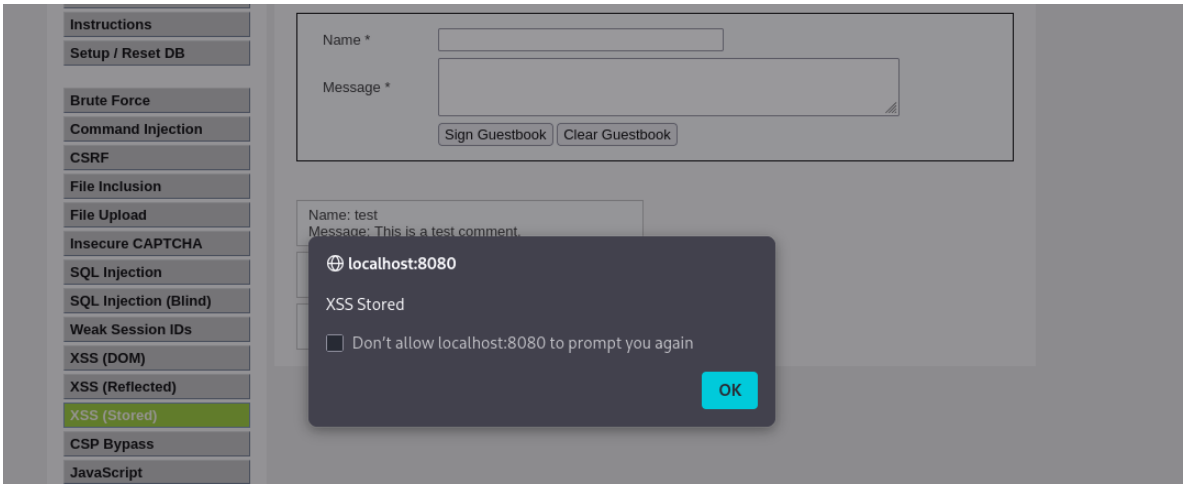
Prueba normal (comportamiento esperado)



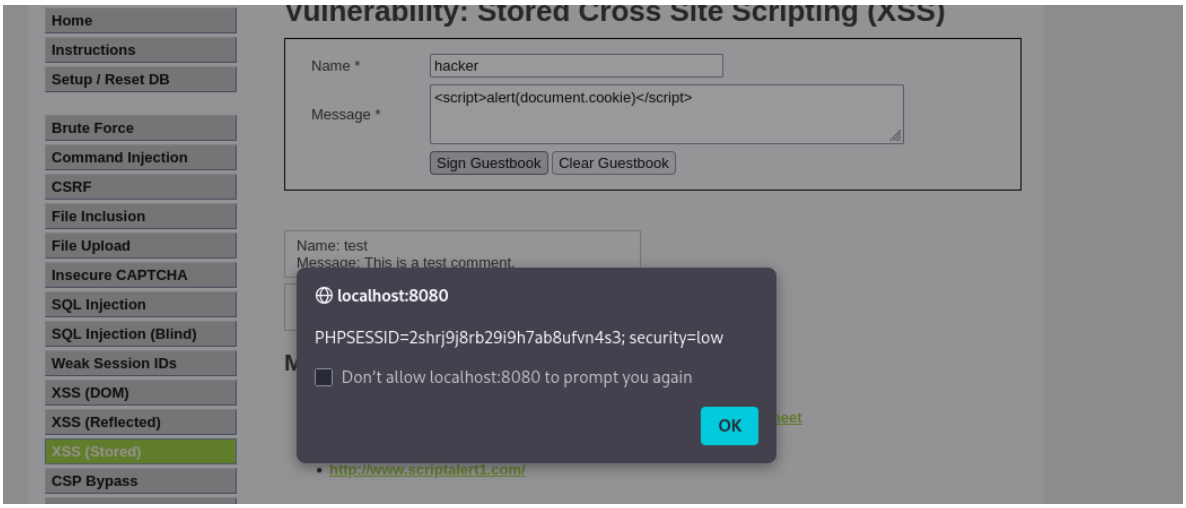
Inyección XSS almacenada



Validación (clave del laboratorio)



Pruebas más avanzadas



Conclusión técnica: Las pruebas del libro de visitas evidencian que el payload malicioso se almacenó de forma permanente en la base de datos y se refleja en el frontend cada vez que un usuario accede a la sección. Esta vulnerabilidad ocurre por una falla en dos puntos de la arquitectura: el backend recibe y guarda la cadena contaminada sin validarla previamente, y el frontend extrae esos registros y los renderiza sin codificarlos de manera segura.

Mitigación: Abordaría esta corrección aplicando una defensa en profundidad. Primero, realizaría una sanitización de entrada en el servidor al momento de recibir el formulario, limpiando las etiquetas peligrosas. Sin embargo, como medida principal y definitiva, garantizaría que cualquier dato proveniente de la base de datos pase por un filtro de codificación contextual estricta antes de ser inyectado en el HTML. De este modo, si algún dato malicioso lograra filtrarse a la base de datos, perdería toda capacidad de ejecución al ser presentado al usuario.